



**DEPARTMENT OF ELECTRICAL AND
COMPUTER ENGINEERING**

PROJECT REPORT

Submitted by:

Name: Eusra Amreen

Id: 2321505642

Course: CSE 115

Section: 02

Submitted to:

Md. Abdullah Al Sayed

Dr. Nova Ahmed

Agriculture Management System

INTRODUCTION:

The code represents an Agricultural Management System implemented in C. This system is designed to facilitate the management of information related to multiple farms, allowing users to perform various tasks such as adding new farm records, searching for farms, deleting farms, and viewing farm data. Additionally, it includes user registration and approval functionalities to control system access.

OBJECTIVES:

The objectives of the Agricultural Management System are to provide a comprehensive and efficient platform for managing agricultural data. Here are the main objectives of the system:

1. Farm Data Management:

To facilitate the systematic recording and management of farm-related information, including details about crops, livestock, production quantities, and sales.

2. User Authentication and Authorization:

To ensure secure access to the system, allow only authorized users to log in and perform actions based on their roles (admin or regular user).

3. User Registration and Approval:

To enable new users to register for the system, with admin approval required for access. This ensures controlled user management and system security.

4. Data Persistence:

To implement file operations for reading and writing farm data to a file. This ensures that farm data is persistent across different sessions of the program.

5. Search and Retrieval:

To provide users with the ability to search for specific farms by name, facilitating easy retrieval of detailed information about individual farms.

6. Farm Deletion:

- To allow authorized users (admin) to delete farms from the system, ensuring data integrity and the ability to manage the list of farms effectively.

7. User-Friendly Interface:

- To offer a user-friendly interface with graphical elements and a menu-driven system. This enhances the user experience and makes the platform accessible to individuals with varying levels of technical expertise.

8. Approval Workflow:

- To establish a workflow where user registrations require admin approval. This feature ensures that only authorized individuals have access to the system.

9. Data Presentation:

- To present farm data in a structured and readable format, making it easy for users to view and analyze information about different farms.

10. System Extensibility:

- To design the system in a way that allows for future expansion and additional features. This includes accommodating more farms, users, and enhancing functionality based on evolving requirements.

11. Contributor Acknowledgment:

- To acknowledge and credit contributors and evaluators of the software, fostering a collaborative and appreciative development environment.

Overall, the Agricultural Management System aims to streamline farm data management, enhance user access control, and provide a user-friendly platform for efficient interaction with agricultural information.

LIBRARIES USED:

`stdio.h` : It is a built-in header file in C. The acronym stdio stands for Standard Input and Output. This header file contains 3 variables, multiple function definitions, and multiple macro definitions.

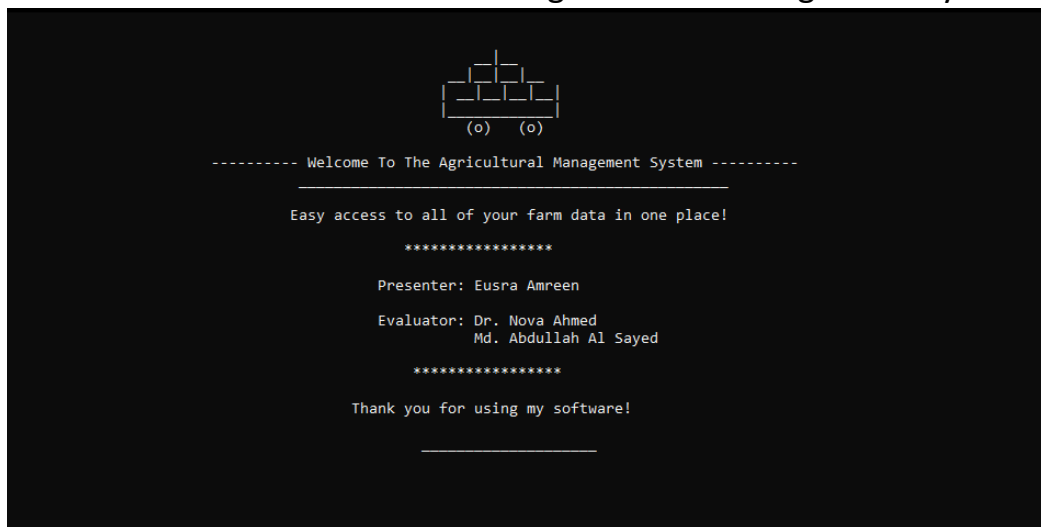
stdlib.h : stdlib. h stands for Standard Library. It has information related to input/output functions. It has information related to memory allocation/free functions.

string.h : It is a standard header file in the C language that contains functions for manipulating strings (arrays of characters). <string. h> header file contains some useful string functions that can be directly used in a program by invoking the #include preprocessor directive.

FUNCTIONS USED:

void interface():

- This function prints a graphical representation of a tractor and provides a welcome message, acknowledging the contributors to the software.
- It serves as an introduction to the Agricultural Management System.



void addFarm(struct farm f[], int *numFarms, int maxSize):

- Allows the user to input details of a new farm, such as farm name, farmer's name, phone number, crop types, production quantity, livestock breeds, and more.
- Checks if the array of farms is full before adding a new farm.

```
Farm details:

Farm name: one
Farmer's name: mr.shoaib
Farmer's phone number: 12345
Types of Crop or Plant: tomato
Production from Crops: 0
Chicken breeds available: local
Cow breeds available: local
Fish breeds available: rui,katla
Milk available: 34
Eggs available: 56
Sales: 78
Chicken available: 4
Cow available: 6
Fishes available: 20
```

void viewFarms(struct farm f[], int numFarms):

- Displays detailed information about all existing farms, including farm name, farmer's name, phone number, crop types, production quantity, livestock breeds, and more.

```
Farm 1:
Farm Name: ne

Farmer's Name: mr.shoaib

Phone: 12345

Crop types: tomato

Production from crop: 0.00 kg
Chicken breeds available: local

Cow breeds available: local

Fish breeds available: rui,katla

Milk available: 34 Liter
Eggs available: 56
Sales: 78.00$
Chicken available: 4 Liter
Cow available: 6
Fishes available: 20
```

void searchFarm(struct farm f[], int numFarms, const char *name):

- Enables the user to search for a specific farm by providing the farm name.
- If found, it displays detailed information about the farm; otherwise, it notifies the user that the farm was not found.

```

Enter your choice: 2
Enter farm name to search: two
Farm found:
Farm Name: wo

Farmer's Name: saqid

Phone: 123456789

Crop types: tomato

Production from crop: 0.00 kg
Chicken breeds available: local

Cow breeds available: local

Fish breeds available: rui

Milk available: 56 Liter
Eggs available: 45
Sales: 67.00
Chicken available: 8 Liter
Cow available: 56
Fishes available: 56

```

void *deleteFarm*(struct farm f[], int *numFarms, const char *name):

- Allows the user to delete a farm by providing the farm name.
- Checks if the user has admin privileges before allowing the deletion.
- Shifts array elements to remove the deleted farm.

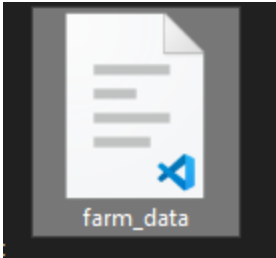
```

Enter farm name to delete: two
Farm deleted.

```

void *writeFarmDataToFile*(struct farm f[], int numFarms):

- Writes farm data to a file (farm_data.dat) to ensure data persistence.
- Uses a file pointer to open the file for writing and writes farm details in a formatted manner.



void readFarmDataFromFile(struct farm f[], int *numFarms, int maxSize):

- Reads farm data from the file (farm_data.dat) when the program starts.
- Populates the array of farms with the data stored in the file.

int userLogin(struct User users[], int numUsers, char *username, char *password):

- Reads farm data from the file (farm_data.dat) when the program starts.
- Populates the array of farms with the data stored in the file.

```
Input log in credentials:
```

```
Username: admin  
Password: admin123
```

void purchaseItems(struct farm *Farm):

- Allows the customers/users to purchase products from the aforementioned farms
- Also calculates total costs for the user/customers.

```
Enter farm name to purchase from: one  
Items available for purchase from farm:  
1. Chickens  
2. Cows  
3. Fishes  
4. Eggs  
5. Milk  
Enter the item number you want to purchase: 1  
Enter the quantity of chickens you want to purchase:
```

STRUCTURES USED:

User :

- Structure of user authentication and authorization.
- Contains username and password strings.
- Also contains an integer value of Admin.

UserRequest :

- Structure of registration into the portal.
- Contains username and password strings.
- Applicable only for new users.

```
1. Approve User Registration Request
2. View User Requests
3. Exit
Enter your choice:
```

farm:

- Contains the structure of farms.
- The primary structure of this platform.

FEATURES:

User Authentication and Authorization:

- The system requires users to log in with a username and password.

<i>Username:</i>	<i>Password:</i>
admin	admin123
user1	user123
user2	user456

-
- User credentials are stored in a structure, including fields for username, password, admin status, and approval status.

Farm Data Management:

- The program uses a structure called `farm` to store information about individual farms, such as farm name, farmer's name, phone number, crop types, production quantity, livestock breeds, and more.
- Users can add new farm records, search for farms by name, delete farms, and view detailed information about each farm.

File I/O Operations:

- The system supports reading farm data from a file (`farm_data.dat`) when the program starts and writing farm data back to the file when needed. This ensures data persistence between program runs.

User Registration and Approval:

- New users can register with the system by providing a unique username and password.
- User registration includes an approval status, and only users approved by an admin can access the system.
- Admins have the authority to approve or disapprove user registrations.

Administrative Controls:

- The system includes administrative controls, with administrators having special privileges such as adding farms and approving user registrations.
- Admins are identified by the `isAdmin` field in the user structure.

User Interface:

- The program has a simple text-based user interface that provides a welcome message and a menu with options for various operations.
- The interface includes a graphical representation of a tractor and a brief acknowledgment of the contributors to the software.

Menu-Driven Operation:

- Users interact with the system through a menu-driven interface where they can choose from different options based on the operation they want to perform.

```
MENUE :  
  
1. Add Farm  
2. Search Farm  
3. Delete Farm  
4. Save Data to File  
5. View Data  
6. Manage Users  
7. logout  
8. Exit  
  
Enter your choice:
```

Error Handling:

- The code includes some basic error handling, such as notifying the user if a farm is not found or if there is an issue with file operations.

Logout and Exit Options:

- Users can log out and exit the program.

Sales Logic:

- There is a purchase logic where the program calculates cost.

DESCRIPTION:

Problem Statement:

You are tasked with developing an Agricultural Management System to assist farmers in managing their farms and potential buyers in purchasing farm products. The system should provide a user-friendly interface and various functionalities for efficient farm management.

Requirements:

1. User Authentication:

- Implement a user authentication system with different roles (admin and user).
- Users should be able to log in, and administrators should have additional privileges.

2. Farm Management:

- Farmers should be able to add details about their farms, including farm name, farmer's name, phone number, crop types, production quantity, chicken breeds, cow breeds, fish types, milk quantity, and eggs available.
- Farms should be searchable, viewable, and deletable.

3. Purchase System:

- Users should be able to purchase farm products such as chickens, cows, fishes, eggs, and milk.
- Implement a mechanism to calculate and deduct the cost of purchased items from the farm's sales.

4. User Registration Requests:

- Users should be able to submit registration requests.
- Administrators should have the ability to view and approve these requests.

5. User Management (Admin Only):

- Administrators should be able to manage farms and users.
- They should be able to add new farms, search for farms, delete farms, view farm data, and manage user registrations.

6. File I/O:

- Implement file input/output to store farm data persistently.

7. Interface:

- Create a user-friendly interface, possibly with ASCII art or other visual elements, to welcome users and guide them through the system.

8. Security Considerations:

- Ensure secure handling of user credentials and sensitive data.

9. Logout and Exit Options:

- Users should be able to log out, and there should be an option to exit the program.

Additional Challenges (Optional):

- Implement advanced user authentication features (e.g., password hashing).
- Enhance the farm management system with more details and options.
- Consider implementing a graphical user interface (GUI) for improved user interaction.

****Note: **** You are encouraged to follow best coding practices, modularize your code, and provide comments for clarity. Additionally, test your system thoroughly to ensure proper functionality.

ERRORS ENCOUNTERED:

- string input errors.

- Statement issues.
- Search printing issues.
- Purchasing malfunction

CONCLUSION:

Expected Improvements:

- Farm capacity might be increased.
- More items might be available for purchasing for customers/user.
- Development in purchasing system.
- A report or review option might be added.
- Security system might be enhanced.
- Bug fixes.